

Blink Qt — Porting Plan, Phase A: Protocol Compatibility

Goal: Blink Qt behaves identically to Blink for macOS and Sylk Mobile **on the wire, in XCAP and in storage**, before any UI work. UI (message pane, bubbles, filter bar, grid, contact preview line) is Phase B and only consumes the APIs built here.

Reference: `blink-macos-messaging-inventory.md` (section numbers below as "Inv §n").

Each numbered step is meant to be one darcs patch (suggested patch name in *italics*), independently testable, and leaves Blink Qt working.

0. Ground rules

1. **Work in the existing Qt file structure.** Existing files (`history.py`, `messages.py`, `contacts.py`, `logging.py`, `configuration/*.py`) are edited; new files are added next to them under `blink/` only where macOS has a self-contained module.
2. **Port, don't re-derive.** Where macOS has pure-Python protocol logic (no AppKit/Foundation), the Qt file is a near-verbatim copy with the same function names and constants, so a later macOS fix can be carried over by diffing the two files.
3. **Fix the macOS protocol defects while porting** (Inv §20 items 1–5, 9, 12, 13) and carry the fixes back to macOS.
4. **Qt keeps its own settings names and DB file;** only the semantics must match. No data migration between Mac and Linux.
5. **Nothing decrypts at intake** (journal or live-without-UI). Decryption happens when something is displayed or previewed.
6. Every step keeps the existing Qt chat window working.
7. Baseline versions: latest python3-sipsimple and SylkServer (both trees available). Outgoing messages are `text/plain` unless the input has formatting. `auto_answer` is a shared contact attribute.

1. Protocol modules and logging (no behaviour change)

1.1 New files ported from macOS

Added Sylk protocol modules ported from Blink for macOS

New Qt file	Ported from (macOS)	Content
<code>blink/pstn.py</code>	<code>util.py</code> <code>pstn_*</code> , <code>canonical_pstn_uri</code> , <code>same_phone_number</code> , <code>normalize_anonymous_uri</code> , <code>pstn_uri_spellings*</code>	Inv §13.3
<code>blink/uris.py</code>	<code>SMSWindowManager._canonical_uri</code> , <code>bare_instance_id</code> , <code>is_placeholder_uri</code> , <code>illegal_uri</code> , <code>isFileableAddress</code>	single canonicaliser (fixes Inv §20 #5)
<code>blink/message_envelopes.py</code>	<code>MessageHost</code> : content-type constants, file-transfer envelope + RCS normaliser, <code>file_transfer_category</code> , <code>label / reply / peaks / call_recording</code> envelopes, <code>conversation_preview</code> , <code>public_key_id</code> , <code>CDR build_call_record / merge_call_records / call_summary</code> ; <code>HistoryManager.classify_category</code>	Inv §2.4, §4.1, §7, §9.1, §12, §17
<code>blink/location.py</code>	<code>SylkLocation.py</code> (already pure)	Inv §11
<code>blink/addressbook_origin.py</code>	<code>AddressbookOrigin.py</code> (already pure)	Inv §13.6

New Qt file	Ported from (macOS)	Content
blink/addressbook_notify.py	AddressbookNotify.py (already pure)	Inv \$14.3
blink/key_escrow.py	KeyEscrow.py (logger swapped)	Inv \$16.1
blink/journal.py	SMSWindowManager journal download/apply (syncConversations, _downloadJournal, _applyCachedJournals, _applyJournalEntries)	\$4 below

Everything else is edits to existing Qt files.

1.2 Tests

Added tests for the ported protocol modules

tests/ with vectors also valid on macOS (and on sylk-mobile where it has the same rule): PSTN inputs → E.164 (incl. 00, trunk zero, 39/378), canonical URIs, category per content type / filename, preview filter, location v1/v2/legacy payloads, CDR merge ranks, modified_hash, addressbook tick encode/decode incl. truncation.

1.3 Activity log

Added Activity log

Qt has a Logs window with SIP / MSRP / XCAP / messaging / pjsip trace tabs but no application-level log like macOS's Activity panel (BlinkLogger → logs/activity.txt + Activity tab). Add:

- ActivityLog singleton in blink/logging.py with info / warning / error / debug, same semantics as macOS BlinkLogger: always written to <logs.directory>/activity.txt (UTF-8, errors='replace', own lock, reopen on error), mirrored to stdout, fanned out to the window. Debug lines only when a debug setting is on.
- **Activity** tab first in logs_window.ui; lines buffered and flushed every 150 ms (cap 4000 pending, count dropped lines); in-memory backlog capped at 5000 while the window is closed; detached when hidden so a journal burst does not flood the GUI thread.
- Prefixes so the file can be grepped per subsystem: [journal], [sync], [db], [ab], [ab] [origin], [ab] [repair], [ab] [notify], [escrow], [Message with <uri>].

1.4 Import assessment logging

Log journal and addressbook import statistics

For repeated from-scratch runs (10K journal entries, 100 contacts) every run must be comparable. Log:

Stage	Lines
Startup	DB file, schema version before/after, each migration step with rows touched and duration; totals per table
Journal start	account, URL, cursor or since=, server version (from response header if available)
Each page	entries, bytes (compressed/uncompressed), fetch time, cached file name, new cursor
Apply progress	every 250 entries: done/total, entries/s
Per run summary	per content type: received / stored / duplicate / skipped (IMDN on first sync, own echo, ignored type) / failed; pending removals deferred and resolved; unread counted; conversations touched; download and apply durations
Per contact summary	top 50 contacts: messages per content type, unread, first/last timestamp; UNHANDLED line with every unknown content type and count
DB check after run	count(*) per category, per content_type, unread total, tombstoned total — must match the run summary
XCAP reload	account, ETag, contacts, groups, members per group (dump_group_members), origin diff ("by <device> at <time> (<reason>)")
Healing	one line per action with reason (conference-domain, e164, echoed-name, dup-uri, file-into-kind-group), MERGE keep id=... drop=..., kind stamps, ensured groups, Messages audit (added/removed)
Notifications	ticks suppressed / sent / received / fetched, with ids or truncated

Stage	Lines
Escrow	restore / repair / refusal reason

Also write the run summary as JSON (`logs/import-<account>-<timestamp>.json`) so two runs can be diffed.

1.5 Reset script

Added script to reset Blink state for import tests

`scripts/reset_state.sh <account>`: removes `message_history.db`, `journal/`, `file_transfers/`, `addressbook_origins/`, `downloads`, the XCAP document cache for the account, and clears `history_synchronization_id` / token; keeps account credentials and keys (flag `--keys` to remove keys too, for escrow tests).

2. Qt pre-fixes that block compatibility

Fix SIP MESSAGE failure reporting and history sync IMDN

Fix	Where
OutgoingMessage._NH_SIPMessageDidFail returns early whenever <code>__disabled_imdn_content_types__</code> is non-empty (always) → failures never surface, retry never triggers	<code>messages.py:540</code>
if <code>['direction'] == 'incoming'</code> always false → no delivered IMDN from sync	<code>messages.py:990</code>
Call history stored as <code>str(list)</code> and rendered with <code>eval()</code>	<code>history.py</code> , <code>chatwindow.py:3466</code> → JSON
<code>MessageHistory.remove(account)</code> uses <code>account=</code> instead of <code>account_id=</code>	<code>history.py:995</code>
Migration chain: each version step handles only one hop	<code>history.py:check_table_version</code> → sequential if <code>version < n</code> steps

3. Storage

3.0 Schema review (do first)

Compared: macOS `HistoryManager.py` (`history.sqlite`: `versions`, `sessions`, `chat_messages v22`, `file_transfers v2`) against Qt `history.py` (`message_history.db`: `table_versions`, `messages v4`, `downloaded_files v1`; call log pickled to `calls_history`). Then cross-checked with `sylk-mobile` (below).

Tables

macOS table	Qt equivalent	Verdict
<code>chat_messages</code>	<code>messages</code>	Extend in place — missing columns below
<code>versions</code>	<code>table_versions</code>	OK
<code>sessions</code> (local call log, source for CDR migration and Calls ordering)	none — pickle file <code>calls_history</code> + <code>application/blink-call-history</code> rows in <code>messages</code>	Not needed as a table. Call history becomes CDR rows in <code>messages</code> (<code>content_type='application/blink-call-detail-record'</code> , <code>category='call'</code>); migrate the pickle and the old <code>blink-call-history</code> rows into CDR rows (macOS did the same in v20). Last-call times derive from those rows.
<code>file_transfers</code> (legacy MSRP transfer log, unchanged since July)	<code>downloaded_files</code>	No new table. HTTP transfer state lives in the stored envelope (error field) and the file cache, as on macOS. Keep <code>downloaded_files</code> .
—	—	New: pending_removals (<code>message_id</code> , <code>account_id</code> , <code>remote_uri</code> , <code>removed_at</code> , <code>source</code>) — removal notices whose target has not arrived yet (fix for Inv \$20 #3, macOS lacks it too)

messages columns

macOS chat_messages	Qt messages	Needed for	Action
msgid	message_id	—	OK
local_uri	account_id	—	OK
remote_uri	remote_uri	—	OK (Bonjour key change §3.3)
time	timestamp	—	OK
date	—	daily index / jump to date	compute date(timestamp) in SQL; no column
direction, body, content_type, status, encryption	direction, content, content_type, state, encryption_type	—	OK
cpim_from, cpim_to	uri, display_name (sender only)	replicated-message attribution, Bonjour peer name, info panel	add cpim_from, cpim_to
cpim_timestamp	—	sender time vs receive time, edit ordering, info panel	add
media_type (sms, chat, audio, video, missed-call, audio-recording...)	— (implied by content type)	last-message vs last-call times, CDR media	add
sip_callid	—	CDR merge key (account, sip_callid), info panel	add
sip_fromtag, sip_totag	—	CDR detail	skip (inside CDR metadata)
private	—	OTR / conference private messages	add (cheap; macOS renders private fill)
uuid	—	legacy	skip
journal_id	—	journal dedup / info panel	add
read	derived from state	persisted unread	add , INTEGER DEFAULT 1
metadata	—	location v2, CDR JSON, CPIM app.Metadata	add , LONGTEXT
related_msg_id, related_action	—	location trails, sidecars (reply, label, peaks), removals	add
category	—	filter bar, previews	add
expire_time	—	reserved (mobile expire)	add , INTEGER DEFAULT 0
— (mobile has_link)	—	Links filter in SQL	add , INTEGER
deleted, deleted_time	state='deleted'	tombstones, revival	add ; keep state untouched
—	decrypted, decryption_error	Qt's deferred-decryption bookkeeping	keep (macOS uses encryption + rewrite)
—	disposition	IMDN request	keep

Indexes

macOS	Qt	Action
unique (msgid, local_uri, remote_uri)	unique (message_id, account_id, remote_uri)	OK
msgid, remote_uri	same	OK
local_uri	—	add
(remote_uri, time)	—	add (paging)

macOS	Qt	Action
(remote_uri, category, time)	—	add (filtered paging)
mobile (... , category, has_link, time)	—	add (remote_uri, category, has_link, timestamp)
read	—	add
related_msg_id	—	add

Cross-check with `sylk-mobile` (`app/app.js createTables`, messages table v20)

macOS took `related_msg_id`, `related_action`, `category`, `metadata`, `deleted` from mobile; `expire` is mobile's name (macOS/Qt use `expire_time` because `SQLObject` reserves `expire`). Mobile columns with no macOS equivalent:

Mobile column	Meaning	Qt
<code>has_link</code> INTEGER (+ index <code>account</code> , <code>from_uri</code> , <code>to_uri</code> , <code>category</code> , <code>has_link</code> , <code>unix_timestamp</code>)	text contains a link; makes the Links filter SQL-pageable (macOS uses a body like probe and narrows client-side)	add — set at insert and on decrypt
<code>content_encrypted</code> BLOB	original ciphertext kept after decryption (COALESCE on update)	skip (decided) — plaintext replaces content after decryption, as on macOS
<code>origin</code> TEXT	originating device (CDR <code>local.deviceId</code>)	skip; metadata carries it
<code>call_id</code>	SIP Call-ID	= <code>sip_callid</code> above
<code>pinned</code> , <code>expire_interval</code> , <code>system</code> , <code>pending</code> , <code>sent/received</code> (+ timestamps), <code>url</code> , <code>local_url</code> , <code>image</code> , <code>sender</code> , <code>disposition_notification</code>	mobile UI/state	skip (Qt <code>state</code> , <code>disposition</code> , file cache cover them)

Mobile also keeps a local `contacts` table (last message, unread, `deleted` / `deleted_timestamp` for two-stage delete, `last_call_*`) and `contacts_groups`. Qt and macOS get the same from the sipsimple addressbook plus history queries, so **no contacts table**.

Non-SQL state macOS keeps (Qt needs the same, files under `ApplicationData`): `journal/<account>/` page cache, `addressbook_origins/<account>.json`, `bonjour_neighbours.json` (local renames), `file_transfers/<account>/<peer>/<transfer_id>/` cache. In-memory only on macOS and fine to keep that way: unread counts (seeded from `read`), previews, seen-id ring, location sharing state, composing state.

3.1 Schema v4 → v5

Extended message history schema for categories, read state and tombstones

One migration step adding every column marked **add** above plus the indexes, then:

Column	Backfill
<code>read</code>	0 for incoming state != 'displayed' rows whose content type is renderable (text/*, file transfer); fixes stray api/pgp rows inflating counts
<code>category</code>	text via one UPDATE; file transfers classified in Python in chunks of 500, explicit commit, re-count
<code>deleted</code> , <code>deleted_time</code>	<code>deleted=1</code> where <code>state='deleted'</code>
<code>media_type</code>	sms for text/file rows, <code>call</code> rows from CDR conversion
<code>cpim_from/cpim_to</code>	from uri / account / remote by direction
<code>call history</code>	pickle + blink-call-history rows → CDR rows (<code>source='migrated'</code>)

Create `pending_removals`.

Rules copied from macOS `add_message` (Inv §8): coerce content type to `str`; classify at insert unless given; drop own self-transfer echo; on duplicate update only `state` and `journal_id`; CPIM timestamp → naive UTC with fallback. **Stop dropping non-text/* content types** (`messages.py:1384`): unknown types are stored inert (`read=1`, no category) — the renderer allow-list decides display later.

3.2 History API

Added message history queries used by sync, contacts and UI

On MessageHistory (db thread, Deferred-returning like today):

- `mark_conversation_read(account, remote)`, `unread_counts(accounts)` (incoming, `read=0`, not deleted)
- `tombstone_message(msgid)` (also `related_msg_id` rows and metadata sidecars by `"messageId": "X"`), `tombstone_conversation(account, remote, before_time, when)`, `restore_conversation`, `deleted_conversations()`
- `delete_message(msgid)` (+ downloaded file), `delete_messages(remote)` — no VACUUM per call (Inv §20 #12); vacuum on idle/startup
- `update_message_body(msgid, body, merge=fn)` (read-modify-write in db thread; location trails)
- `update_decrypted_message(msgid, plaintext)` (stamps missing category)
- `last_message_times()`, `last_message_accounts()`, `last_text_messages(n=5)` (window function, fallback) — for contact ordering and previews
- `present_categories(remote)`, `renderable_cutoff(remote, n, before, category)`, `get_messages(remote, since, before, category, limit)`, `related_messages(msgids)` — paging for Phase B
- `move_conversation(old, new)` — Bonjour re-key

3.3 Bonjour keys

Key Bonjour conversations by bare instance id

Migrate `remote_uri = '<instance_id>@local'` → bare `<instance_id>` (strip `urn:uuid:`), `account_id='bonjour@local'`; move `downloads/<id>/` peers accordingly. Same key as macOS so shared `uri.py` works unchanged (Inv §10).

3.4 Presence rows

Do not store presence changes in message history

Delete existing presence/availability rows; never store new ones.

4. Journal (SylkServer) rewrite

4.1 Token and triggers

Rework history token handling

- `application/sylk-api-token` rate-limited 30 s per account, not gated on replication (uploads need it); 401 → re-request after 30 s.
- Pass `tls_name` into the proxy lookup for token/API messages.
- Sync on registration success (+10 s), token change, replication enabled. Drop the 500 s runtime throttle; replace with the per-account in-progress guard (released in `finally`).

4.2 Paged download to disk

Download the journal in pages and cache them before applying

- First sync (no `history_synchronization_id`): GET `<url>?since=<now - 5 years>` (ISO ms + Z); otherwise GET `<url>/<cursor>`. Authorization: Apikey, gzip, 20 s timeout. Loop until empty, cap 200 pages per run.
- Each page written to `ApplicationData/journal/<account>/<ts>-<lastid>.json` (`{"cursor", "messages"}`); cursor advanced **only after the write**.
- Relies on SylkServer honouring `since` (latest SylkServer; older servers return 3 days).

4.3 Apply stage

Apply cached journal pages with the shared dispatch rules

- Files applied in order; **a file is deleted only after a successful apply** (Inv §20 #2); a failed file is retried next run, with a per-file attempt counter and quarantine after N failures.
- Bulk mode: no sessions/windows created, notifications coalesced, progress log every 250, throttle 50 ms per chunk.
- Dispatch exactly as Inv §6.3, using the constants in `blink/message_envelopes.py`. IMDN payload parsed as JSON/`ast.literal_eval`, **never eval** (Inv §20 #1). IMDN skipped on first sync.
- **No decryption**. Text and location bodies stored as ciphertext with `encryption`; notability from cleartext envelope; location rows get `related_*`, `category`, `metadata` from `blink.location.envelope_summary`.

- Message removals whose target is not present yet are kept in a pending table and re-applied when the target arrives (Inv §20 #3).
- A journalled message never creates a session; it is persisted (unread if notable and not displayed) and announced with a notification for the UI.
- First sync on a device: promote Messages group (4 → §6.3), send "Account activated on <UA>" once (sms.activation_announced equivalent).
- End of run: one BlinkJournalApplied notification with per-contact new-message counts (UI decides banners later).

5. Live message intake and sending

5.1 Unified intake

Route live messages through the journal dispatch

MessageManager._NH_SIPEngineGotMessage uses the same dispatch table as §4.3 for storage; only presentation differs. Adds: seen_message_ids ring (10000) shared with the journal "already taken in" check; sidecar metadata (reply, label, peaks, call_recording) stored read=1, never notify; location stored + trail merged; CDR accepted only from own account and not from this device; unknown types stored inert.

5.2 Read markers

Synchronise conversation read state like Blink for macOS

- Send application/sylk-api-conversation-read {"contact","device_id"} to **own account** (not the peer — Inv §20 #10) on a real unread→read transition; replication on; never Bonjour.
- Incoming application/sylk-conversation-read: **persist** via mark_conversation_read (today only in-memory), clear counts, update open sessions.
- Own echo swallowed silently (device_id match, else 30 s TTL).
- Unread counts keyed by canonical URI (Bonjour: instance id), seeded from unread_counts() at launch.

5.3 Removal

Tombstone removed messages and conversations

- Incoming sylk-message-remove → tombstone target id + sidecars.
- Incoming sylk-conversation-remove → tombstone rows older than the CPIM/entry timestamp; file contact under Deleted (§6.6); a newer message revives.
- Outgoing conversation remove = sylk-api-conversation-remove {"contact","timestamp"} to own account(s); own echo swallowed (60 s).
- Delete-for-both on Bonjour: peer-to-peer application/sylk-message-remove JSON.

5.4 Sending rules

Align outgoing CPIM/IMDN rules with Blink for macOS

No CPIM for sylk-api-* and text/pgp-public-key; renderable messages carry Message-ID + Disposition-Notification: positive-delivery, display; OTR messages X-Sylk-Skip-Journal: yes; read app.Metadata CPIM header into metadata; Bonjour adds instance_id to From. Is-composing: ignore own replicated echo; send with refresh 60. Public key lookup once per (account, uri) per launch, only when the user sends.

Qt currently sends every message as text/html; send text/plain unless the input has formatting (decided).

6. Contacts and XCAP

6.1 Shared attribute namespace

Share contact attributes with Blink for macOS and Sylk Mobile

- SharedSetting.set_namespace('ag-projects:sipsimple') (Qt uses ag-projects:blink). One-time read fallback from the old bag for preferred_media, written back to the new bag.
- Add the macOS extensions: contact shared organization, auto_answer, preferred_media, disable_smileys, modified_*; local chat_language, silence_notifications, no_mediaproxy, public_key, public_key_checksum; group shared kind + modified_*; URI shared position.

- `auto_answer` becomes a `SharedSetting` (local in Qt today); read the old local value once and write it to the shared bag.

6.2 Device stamps

Stamp contact and group changes with the device that made them

`blink.addressbook_origin.install(Contact, Group, device_id=bare instance id, agent=UA, log=...);` reason stack used by every automatic writer (`repair`, `call-history`, `backfill`, `group-kind`, `ensure-group`, `file-into-kind-group`, `block`); origin diff logged on each XCAP reload; snapshot in `ApplicationData/addressbook_origins/`.

6.3 Reserved groups as real XCAP groups

Use the shared Messages, Calls, Tel and Deleted groups

- Resolve by kind → name → reserved id: `_messages`, `_deleted`, `_calls`, `_tel`, `_blocked`, `_conference`, `_favorites`. `stamp_group_kinds` fills missing kinds only. `ensure_group` only once XCAP is loaded (or not expected).
- Qt's virtual `MessageContactsGroup` (`__messages`) becomes a view over the real `_messages` group; `interface.show_messages_group` only controls visibility.
- Messages membership: filed from history (`auditMessagesGroupAgainstHistory`), saved only on change, never Bonjour/placeholders; promoted to top on creation and after first journal sync.
- Messages, Calls, Tel, Deleted: users cannot add/remove members (enforced in the model, not just the editor — Inv §20 #6).

6.4 Canonical URI and PSTN

Normalise phone numbers to E.164

- Add account `psn.replace_leading_zero` (Qt has `idd_prefix`, `prefix` only).
- All conversation keys, unread keys, contact matching and Messages dedup use `blink.uris.canonical_uri` (one function).
- New contacts created from messages/calls store numbers bare E.164, type `tel`.

6.5 Healing pass

Repair and merge contacts after the addressbook is loaded

After `XCAPManagerDidReloadData` + 15 s, once per process, with addressbook notifications suppressed: conference domain fix → E.164 → echoed-name replacement → in-contact URI dedup → file into Tel/Conference. Then, notifications resumed: `mergeMessagesGroupDuplicates` (union-find, lowest id survives, name/URI donation). Same order and reasons as macOS (Inv §13.4–13.5). Not run on the cached document at launch.

6.6 Deletion semantics

Two-stage contact delete and remote deletion rules

- Remote deletion (`data.remote`) keeps history, files, key; local deletion purges only unclaimed canonical URIs (never Bonjour keys).
- Stage 1 Delete: tombstone conversations, move to `_deleted`, out of `_messages`. Stage 2 Delete Permanently: restore keys still claimed, delete contact from XCAP, purge, announce conversation removal.
- Minimal UI hooks only (existing menu actions call the new model methods); full UI in Phase B.

6.7 Addressbook change ticks

Notify other devices when the addressbook changes

`application/sylk-addressbook-update` send/receive with `blink/addressbook_notify.py` (debounce/fuse/backoff constants, X-Sylk-Skip-Journal: yes, truncation semantics). Not sent for remote-applied documents, during healing, or for Bonjour. Receive triggers a forced `resource-lists` fetch with ETag cleared.

6.8 Bonjour guards

Never write Bonjour neighbours to XCAP

No contact creation, key filing, Messages filing or purge for instance-id keys; neighbour renames stored locally. Manual merge of a Bonjour row into an XCAP contact must not add the instance id to XCAP (Inv §20 #8).

Baseline: latest python3-sipsimple, which provides `data.remote` on addressbook notifications and `is_applying_remote_document` on XCAP changes.

7. Keys, files, side channels

7.1 PGP key escrow

Escrow the PGP private key in the own XCAP contact

blink/key_escrow.py after §6.1. Restore on reload when no local key; repair missing escrow once per session; refusal rules as macOS. Map paths: Qt keys/private/<account>.privkey vs macOS keys/*.privkey. Key generation waits for the escrow answer (XCAP loaded / disabled / 15 s). Replaces the generate-or-import prompt when an escrowed key exists.

7.2 HTTP file transfer upload

Upload files over HTTP like Sylk Mobile and Blink for macOS

Qt uploads over MSRP today, which mobile does not consume. Implement: base URL from sms.file_transfer_url (learned from first incoming transfer) or derived from the history URL;

<base>/<sender>/<receiver>/<transfer_id>/<filename>; PGP .asc to peer + self when ≤ 50 MB; Apikey auth, 401 → token; envelope application/sylk-file-transfer; cache

ApplicationData/file_transfers/<account>/<peer>/<transfer_id>/. MSRP stays as fallback (Bonjour, no server).

Download failure classes and error field written into the stored envelope; 404/410 → delete locally if not on disk.

7.3 Sidecars and location storage

Store captions, replies, waveforms and location trails

Label / reply / peaks / call-recording metadata linked via related_msg_id; location trails merged with

blink.location.merge_location_bodies (≤ 1000 points). Rendering in Phase B.

7.4 Call Detail Records

Store and merge call detail records

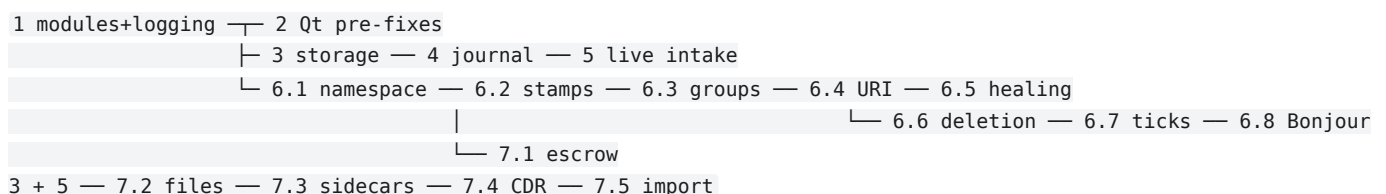
Accept application/blink-call-detail-record (own account, not this device), merge by (account, sessionId) with source ranks; write local calls as CDR rows (source='local') replacing the str(list) call history entries; category call.

7.5 Sylk data import (optional, last)

Import messages and files from a Sylk Mobile export

DataImport protocol client (pure); triggered only by a fresh sylk-data-export announcement from own account; add-only; messages + files.

8. Order and dependencies



Suggested batches:

Batch	Steps	Result
A1	3.0 review, 1, 2, 3	Schema agreed, ported modules, Activity log + import logging, fixed Qt bugs, schema ready
A2	4, 5	Journal and live traffic stored exactly like macOS; read/remove sync across all clients
A3	6.1-6.4	Same XCAP document semantics as macOS and mobile
A4	6.5-6.8, 7.1	Healing, deletion, ticks, escrow converge across devices
A5	7.2-7.5	Files, sidecars, CDRs

9. Verification matrix

Run with one account on Blink Qt, Blink macOS and Sylk Mobile simultaneously, plus a fresh Qt install.

Scenario	Expected
Fresh Qt install, account with years of history	Journal pages cached and applied; history back 5 years; nothing decrypted at intake; no conversation sessions opened; correct unread counts
Read on mobile	Qt unread clears and stays cleared after restart; Qt sends nothing back
Read on Qt	macOS and mobile clear; Qt ignores its own echo (no log line)
Delete-for-both on Qt / macOS	Target tombstoned on all; downloaded file removed on the deleting device
Conversation remove on mobile, then new message	Qt files contact under Deleted, hides older rows; new message revives
Two-stage delete on Qt	Stage 1 visible as <code>_deleted</code> on macOS/mobile; stage 2 removes contact from XCAP and history everywhere
Duplicate contacts in Messages	All three clients converge on the lowest id
Phone number in national / 00 / + form	One conversation key, one contact, E.164 stored
Edit contact on Qt	<code>modified_by</code> = Qt instance id; macOS logs "by <Qt device>"; addressbook tick sent once; macOS fetches
New device with escrowed key	Qt restores the key without prompting
File from Qt to mobile (encrypted)	Mobile downloads and decrypts; journal echo merges into Qt row
Caption / reply / location / CDR from mobile or macOS	Stored and linked in Qt (visible in Phase B)
Bonjour chat Qt ↔ macOS	Same instance-id key; nothing written to XCAP
Golden vectors	Pass on both trees; sync check shows no drift

10. Decisions

#	Question	Decision
1	Code organisation	Existing Qt structure; new files only for self-contained macOS modules, ported near-verbatim (§1.1)
2	Qt schema	Extend <code>messages</code> in place (§3)
3	<code>text/html</code> vs <code>text/plain</code>	<code>text/plain</code> unless formatted
4	<code>auto_answer</code>	Shared
5	macOS protocol defects	Fix while porting, carry back to macOS
6	Versions	Latest python3-sipsimple and SylkServer
7	Keep ciphertext after decryption (<code>content_encrypted</code> , like mobile)	No — plaintext replaces <code>content</code> , as on macOS